
Assignment Report: The Greatest Adventurer Game

1 Introduction [2']

This assignment, focuses on implementing a multithreaded game program using the Pthread library in Linux. The objective is to design and develop a simple console-based game called “The Greatest Adventurer”, where an adventurer moves around a dungeon to collect gold shards while avoiding moving walls.

2 Design [5']

The overall design of this program follows a modular and multithreaded architecture, ensuring clear separation between game logic, user input, object movement, and rendering. The goal was to build a responsive and thread-safe console game that meets all assignment specifications.

2.1 Program Structure

The program is divided into several key components:

- **Main Thread** – Handles user input (W, A, S, D, and Q) and updates the player’s position accordingly.
- **Wall Threads** – Six threads control the motion of six independent walls that continuously move left or right, reappearing from the opposite side when reaching a boundary.
- **Gold Shard Threads** – Six threads control the random motion of gold shards, which disappear once collected by the player.
- **Render Thread** – Responsible for continuously redrawing the dungeon map to display real-time updates of player movement, wall positions, and gold shards.

2.2 Thread Synchronization

Since multiple threads modify shared data (the map, player, and objects), mutual exclusion (mutex) is implemented using `pthread_mutex_t`. The mutex ensures:

- No race conditions occur during map updates.
- Player, walls, and shards always have consistent positions.
- Collisions are detected accurately without graphical errors.

2.3 Game State and Control Flow

A global variable `game_state` is used to track whether the game is running, won, lost, or exited. Each moving object thread checks this variable periodically to stop gracefully when the game ends. The input thread (main thread) listens for Q to exit or updates the player’s movement direction. When all shards are collected, the player wins; when colliding with a wall, the player loses.

2.4 Timing and Smooth Animation

The movement speed of each object is controlled using `usleep()`:

- Walls move every 0.15 seconds, allowing full traversal within 10 seconds as required.
- Shards move slightly slower at 0.20 seconds for better visibility.
- Rendering occurs every 0.05 seconds to ensure smooth animation.

2.5 Terminal Rendering

ANSI escape codes are used to clear and refresh the terminal screen without residual characters. Keyboard inputs are captured in non-blocking mode using `termios` and `fcntl`, allowing continuous input reading while threads run concurrently.

3 Environment and Execution [2']

3.1 Environment Setup

The program was developed and tested under the following environment configuration:

- **Operating System:** Ubuntu 20.04.6 LTS
- **Kernel Version:** Linux 5.15.10
- **GCC Version:** 9.4.0

This environment is consistent with the course specification and ensures compatibility with the TAs' testing setup.

3.2 Compilation and Execution

The program can be compiled and executed in a Linux terminal using the following commands:

```
g++ hw2.cpp -lpthread
./a.out
```

Once executed, the player can control the adventurer using the following keys:

- W/A/S/D – Move Up/Left/Down/Right
- Q – Quit the Game

3.3 Demo Screenshots

To demonstrate the functionality of the game, three screenshots were captured to show the different game states: **Exit**, **Lose**, and **Win**.

Figure 1: Game Exit Screen


```
问题 输出 调试控制台 终端 端口

|===|=====|
|                                     |
|             =====|
|=====|                                     |=====|
|=====|                                     |=====|
|=====|                                     |=====|
+-----+-----+
a
+-----+-----+
|                                     |
|             0|
|             =====|
|             =====|
|=====|                                     |=====|
|=====|                                     |=====|
|=====|                                     |=====|
|=====|                                     |=====|
+-----+-----+
+-----+-----+
|                                     |
|             0|
|             =====|
|             =====|
|=====|                                     |=====|
|=====|                                     |=====|
|=====|                                     |=====|
|=====|                                     |=====|
+-----+-----+
You win the game!!
❖ csc3150@csc3150:~/project2/source$
```

Figure 3: Victory Screen (Player wins after collecting all gold shards)

4 Conclusion [2']

In this assignment, I successfully implemented a multithreaded console game using the Pthread library in Linux. Through the process, I gained hands-on experience in thread creation, synchronization, and real-time input handling. The project not only strengthened my understanding of concurrency control but also enhanced my ability to design modular and responsive systems. Overall, this

assignment provided valuable insight into how multithreading can be applied to create interactive and efficient programs.